Mawlana Bhashani Science and Technology University

Department of Information and Communication Technology

Santosh, Tangail-1902, Bangladesh

# Telegram protocol: MTProto architecture, security and trade-offs

**Shayla Akter**

ID: IT24022 | Session 2023-24

SUPERVISED BY

**Dr. Nazrul Islam**

Associate Professor

Dept. of ICT, MBSTU

# Presentation Outline

01 Introduction and research problem

02 Related work and research gap

03 Proposed solution and methodology

04 Analysis and results

05 Conclusion

# Introduction and Research Problem

**Telegram protects its messages with a custom protocol, MTProto, rather than a standard one like TLS.**

**01** Close to a billion people use Telegram for private and group chats

**02** Default cloud chats are not end-to-end encrypted; custom cryptography needs independent scrutiny

**03** Objective: explain how MTProto works and evaluate its security design

# Background: What Is Telegram?

## Launched in 2013

Founded by Nikolai and Pavel Durov as a cloud-based messaging service

## Custom protocol

MTProto was designed for speed and security on slow mobile networks

## Open client apps

Official client apps are open source; the server code is closed

## Rich features

Cloud chats, Secret Chats, groups, channels and a Bot API

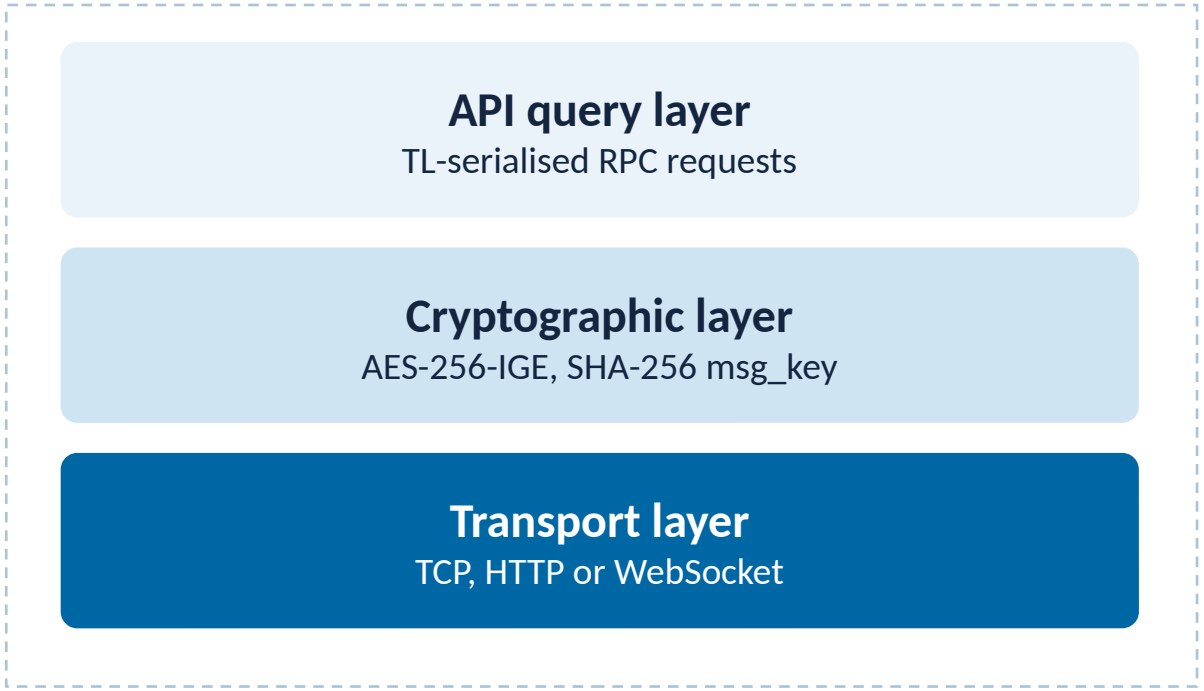
# Related Work and Research Gap

| Study / approach | Key idea | Relevant limitation |
|---|---|---|
| [1] Jakobsen and Orlandi, 2016 | Formal CCA-security analysis of MTProto 1.0 | Found MTProto 1.0 not IND-CCA secure; protocol was revised |
| [2] Miculan and Vitacolonna, 2021 | Automated symbolic verification of MTProto 2.0 | Symbolic model only; does not cover every computational attack |
| [3] Albrecht et al., 2022 | Four attacks and a proof for Telegram | Proof holds for a modified variant, not unchanged MTProto |

**Research gap** Formal proofs cover modified variants, and default chats stay readable by the server.

# Methodology

## MTProto 2.0 architecture




**01** **Step 1: key creation**

Diffie-Hellman exchange makes a 2048-bit auth key

**02** **Step 2: encryption**

Message encrypted with AES-256 in IGE mode

**03** **Step 3: delivery**

Sent over the transport; server decrypts and replies

# Methodology: Key Creation

### 1. Request

Client sends req_pq with a random nonce

→

### 2. Proof of work

Server sends pq and RSA key fingerprints; client factors pq

→

### 3. DH parameters

Server returns prime p, generator g and g^a; client checks them

→

### 4. Shared key

Client replies with g^b; both compute auth_key = g^ab mod p

**Result: a shared 2048-bit auth_key, identified by a 64-bit auth_key_id.**

# Methodology: Message Encryption

## Encrypted packet

| auth_key_id (64 bit) | msg_key (128 bit) | encrypted_data |
|---|---|---|

## Inside encrypted_data

| salt | session_id | message_id | seq_no | length | data | padding |
|---|---|---|---|---|---|---|

- msg_key is the middle 128 bits of SHA-256 over part of auth_key, the plaintext and padding
- AES key and IV are derived from msg_key and auth_key; cipher is AES-256 in IGE mode

# Cloud Chats and Secret Chats

| Feature | Cloud chat (default) | Secret Chat (opt-in) |
|---|---|---|
| **Encryption** | Client to server | End-to-end between two devices |
| **Storage** | On Telegram servers | Only on the two devices |
| **Multi-device** | Yes, syncs everywhere | No, tied to one device |
| **Chat types** | Private, group, channel | One-to-one only |
| **Extras** | Server-side search and backup | Self-destruct timer, key fingerprint check |

Secret Chats use a Diffie-Hellman exchange between the two devices, not the server.

# Analysis Setup

| | |
|---|---|
| **Data / study setting** | Official MTProto 2.0 documentation and published security papers |
| **Baselines / comparisons** | WhatsApp and Signal (Signal Protocol, end-to-end by default) |
| **Evaluation metrics** | Default E2EE, E2EE in groups, open-source server code |
| **Implementation / protocol** | Literature-based review; no new experiments were run |

# Results and Discussion

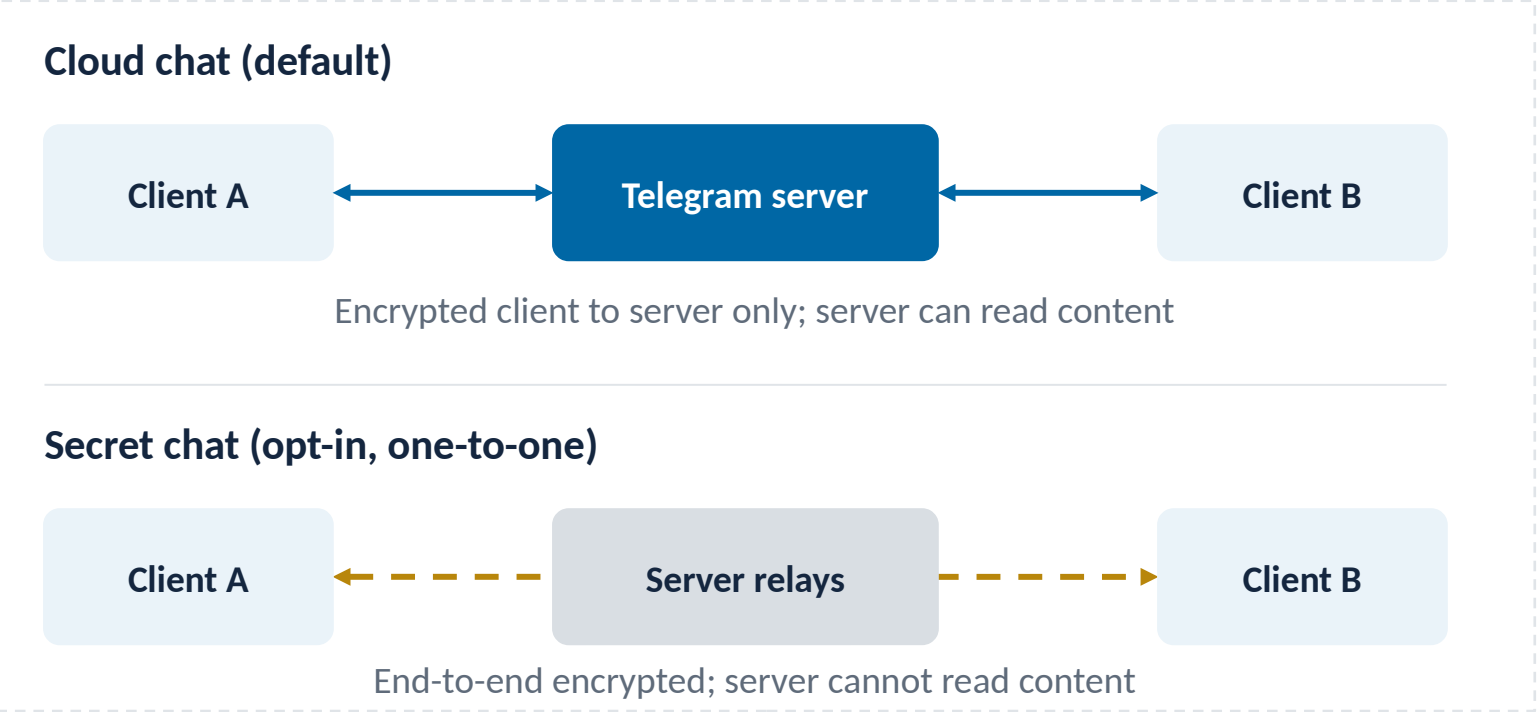



Figure: encryption paths of cloud chats and Secret Chats (Secret Chats work in one-to-one chats only).

## Main finding

Only Secret Chats are end-to-end encrypted.

Cloud chats enable multi-device sync, but the user must trust Telegram's servers.

# Security Properties of MTProto 2.0

| Property | Design in MTProto 2.0 | Note |
|---|---|---|
| **Confidentiality** | AES-256-IGE with per-message keys | Cloud chats stay readable by the server |
| **Integrity** | msg_key binds plaintext and padding | Checked before a message is accepted |
| **Authentication** | RSA key built into official clients | Fingerprint check exists only in Secret Chats |
| **Replay protection** | Unique message_id, session_id, seq_no | Server also checks clock drift |
| **Metadata** | Server sees who talks and when | Not hidden in any chat type |

# Published Security Analyses

| | | |
|---|---|---|
| **2016** | **Jakobsen and Orlandi** | Showed MTProto 1.0 is not IND-CCA secure; MTProto 2.0 later moved to a SHA-256-based msg_key |
| **2021** | **Miculan and Vitacolonna** | Modelled MTProto 2.0 in ProVerif to check secrecy and authentication properties |
| **2022** | **Albrecht et al.** | Reported four attacks, including message reordering; fixes followed, and a proof covers a modified protocol |

**Takeaway: flaws were found and fixed, and the main remaining concern is trust in the server.**

# Quantitative Comparison

| Method | Default E2EE | E2EE in groups | Open-source server |
|---|---|---|---|
| WhatsApp | Yes | Yes | No |
| Signal | Yes | Yes | Yes |
| **Telegram (cloud chat)** | **No** | **No** | **No** |

**Telegram trades default end-to-end encryption for cloud sync and convenience.**

# Cryptographic Primitives Compared

| Method | Key exchange | Encryption and integrity |
|---|---|---|
| **Telegram (MTProto 2.0)** | RSA plus 2048-bit Diffie-Hellman | AES-256-IGE, SHA-256 msg_key |
| **WhatsApp (Signal Protocol)** | X3DH with Curve25519 | Double Ratchet, AES-256, HMAC-SHA256 |
| **Signal** | X3DH with Curve25519 | Double Ratchet, AES-256, HMAC-SHA256 |

**Telegram uses a custom design, while the Signal Protocol is widely analysed and reused.**

# Practical Recommendations

## Use Secret Chats

Choose them for sensitive one-to-one conversations

## Enable two-step verification

A cloud password protects the account if an SMS code leaks

## Review active sessions

Log out devices you do not recognise

## Assume groups are readable

Group chats and channels are not end-to-end encrypted

# Conclusions and Future work

**MTProto 2.0 is a well-studied custom protocol, but its default mode trusts the server.**

**01** Contribution: explained MTProto layers, key exchange and the encryption flow

**02** Limitation: literature-based review without independent cryptanalysis

**03** Next step: test MTProto traffic and Secret Chats in a controlled lab setup

# Selected References

[1] J. B. Jakobsen and C. Orlandi, “On the CCA (in)security of MTProto,” ACM SPSM Workshop, 2016.

[2] M. Miculan and N. Vitacolonna, “Automated symbolic verification of Telegram’s MTProto 2.0,” 2021.

[3] M. R. Albrecht, L. Mareková, K. G. Paterson and I. Stepanovs, “Four attacks and a proof for Telegram,” IEEE S&P, 2022.

[4] J. Sušánka and J. Kokes, “Security analysis of the Telegram IM,” ROOTS, 2017.

[5] Telegram, “MTProto Mobile Protocol,” core.telegram.org/mtproto.

[6] Telegram, “End-to-End Encryption, Secret Chats,” core.telegram.org/api/end-to-end.

# Thank you

# Questions and Discussion

Shayla Akter
Dept. of ICT, Mawlana Bhashani Science and Technology University

ID: IT24022 | Session 2023-24